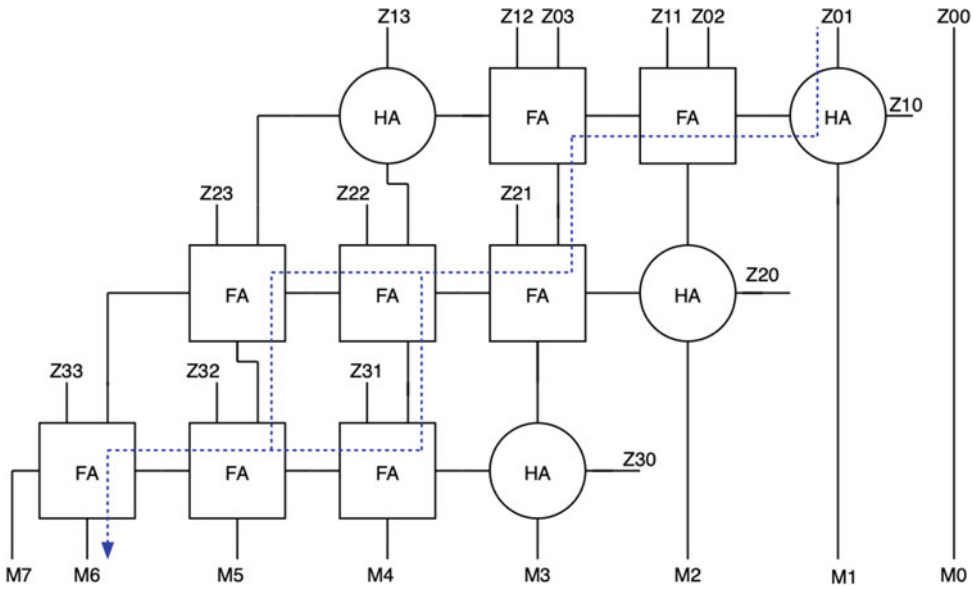

Fig. 11.25 4×4 full multiplier with the critical path(s) shown

only would there not be a unique critical path, there would be many critical paths.

Having non-unique critical paths is both a good sign and a bad sign. On the one hand, multiple equal critical paths mean that for a large subset of paths slack is zero. This is an indicator of a well-designed circuit with optimal power dissipation.

On the other hand, multiple paths with the same slack at the same supply mean that the circuit has little room for improvement. Trying to improve the speed of a combinational block involves trying to reduce the propagation delay of the critical path. Because the multiplier has multiple equal paths, improving any one of them would cause the critical path to migrate to the others. Trying to optimize all paths at the same time is normally paid for in terms of loading on previous stages and total chain effort.

Thus, sizing cannot generally be used to speed up array multipliers. But improving upon multiplier speed is very important, because multipliers tend to dominate the critical path. This can only be done by changes in the algorithmic or architectural level. This can either be done by improving the procedure for combining partial products as in Wallace tree and DADDA multipliers, or by manipulating multiplier and multiplicand strings as with Booth encoding.

11.10 Wallace and DADDA Multipliers

1. Understand that only bit position, not row position is important for partial products
2. View FAs and HAs as 3:2 and 2:2 compressors
3. Use FAs and HAs to maximally cover partial products to design Wallace tree multipliers
4. Understand the complexity of Wallace trees
5. Design DADDA multipliers.

If we examine the process of reducing partial products in Sect. 11.9, we notice that we followed a very restrictive approach towards the reduction. This restrictiveness yields a regular structure where sums and carries move predictably and HAs and FAs are used in well-defined locations. However, the summand reduction process has some wiggle room we have not utilized.

Figure 11.26 shows a more abstract view of partial product reduction. In this view, each partial product is represented by a dot. The parallelogram shape of the partial products is completely unnecessary. As long as the position of each bit is preserved, the operation remains unchanged. Thus, the parallelogram can be reduced to a triangle as shown in Fig. 11.26. This rearrangement is very trivial and has no impact on the result, however, it allows us to focus on reducing the summands.

The task now is to reduce the triangle in Fig. 11.26 into a single $M + N$-bit word that represents the product. This can be done through only two tools: HAs and FAs. HAs accept two bits in the same position and produce two bits, one in the same position and the other in the following position. The FA accepts three bits in the same position and produces two bits, as is the case with the HA, one is in the same bit position, and another is in the following.

HAs and FAs are sometimes called 2:2 and 3:2 compressors, respectively. They combine two or three bits into 2 bits. The multiplication problem can thus be stated as "How can we use 2:2 and 3:2 compressors to reduce the MN partial products into a single $M + N$ word?". This optimization should be done under two constraints area and delay. Using

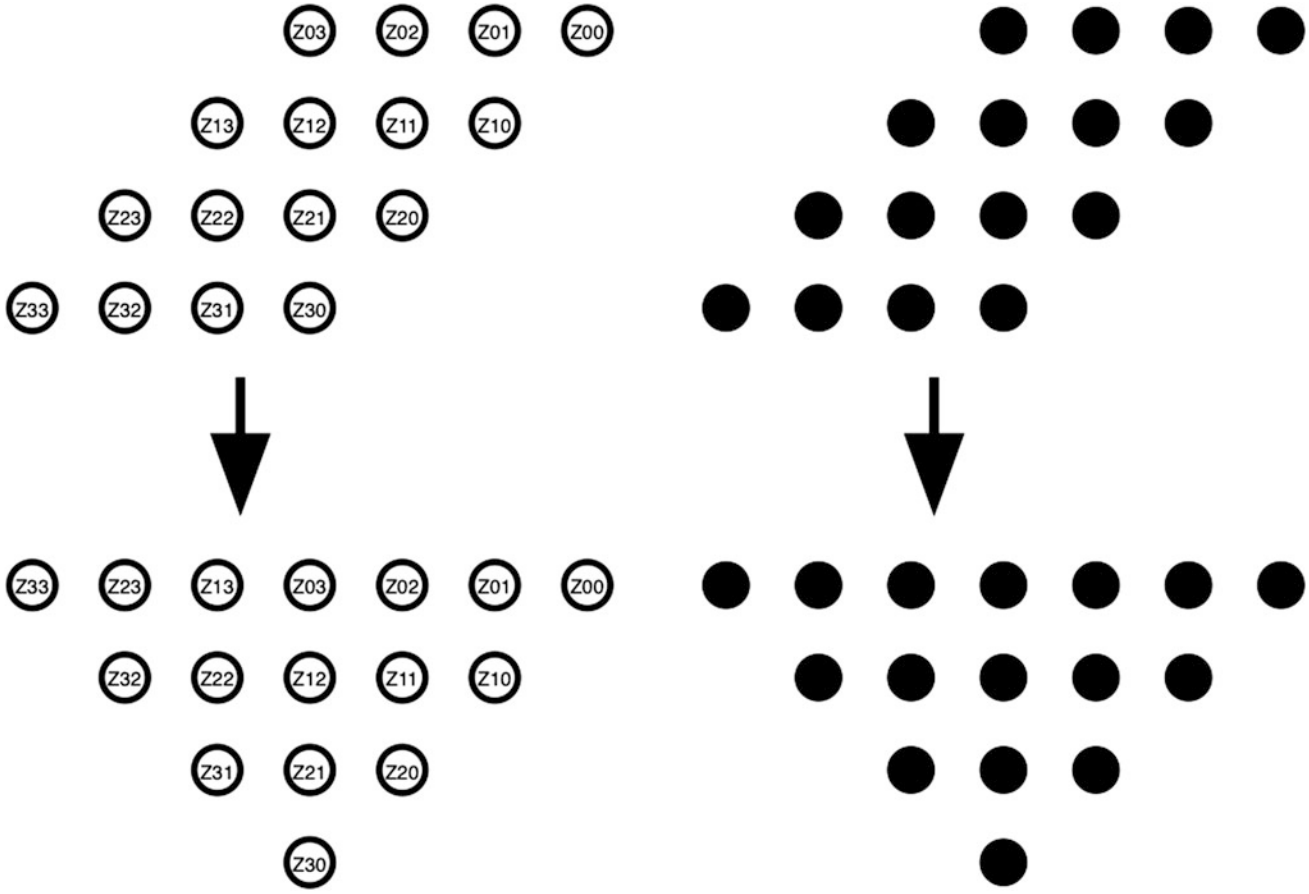

Fig. 11.26 Regular (top) and triangular (bottom) view of partial products

more HAs and FAs means more area is used. Using more of them in cascade means the delay will be longer.

Wallace tree and DADDA multipliers are two (non-unique) approaches used to address this problem. They both reduce the delay of the multiplier relative to Sect. 11.9, especially if it is a very large multiplier. The DADDA multiplier is also very conservative in its use of adders.

Wallace tree adders address the reduction operation by trying to reduce the maximum possible number of bits at every step. Thus if three bits are available in any position, they are compressed using a 3:2 compressor. If only 2 bits are available, a 2:2 compressor is used. The maximum number of units possible are used at every step. This is repeated in subsequent steps until only two bits are left in every location. These can then be added using a fast $N + M$-bit adder.

Formally, the Wallace tree algorithm is as follows:

1. Stop iterating only when 2 or less bits are left in ALL positions.
2. When you stop iterating, feed the resulting two words to a fast adder.
3. While iterating, combine any available three bits in a position using an FA. Combine any available 2 bits using an HA. Use multiple FAs and HAs to achieve maximum coverage in any step. Exceptions in the points below are optional but allow hardware minimization.
4. If any position at any step has only two bits and ALL preceding positions have at most two bits, do NOT use a HA on said position. If the position has three bits, use only an HA.
5. If the final position has only a single bit, do not combine the MSB position or the penultimate positions unless there are either three bits or it is the final step.
6. In the final step use an HA on three bits if the succeeding position has only two bits covered by an HA.

Figure 11.27 shows the Wallace tree for a 4×4 multiplier. In the first step in the top-right corner, the first position with three bits is position 2. An HA is used according to rule 4. In positions 3 and 4 FAs are used according to rule 2. Positions 5 and 6 are left alone according to rule 5.

This leads to the second step in the lower right corner. Position 3 is combined using an HA according to rule 6.

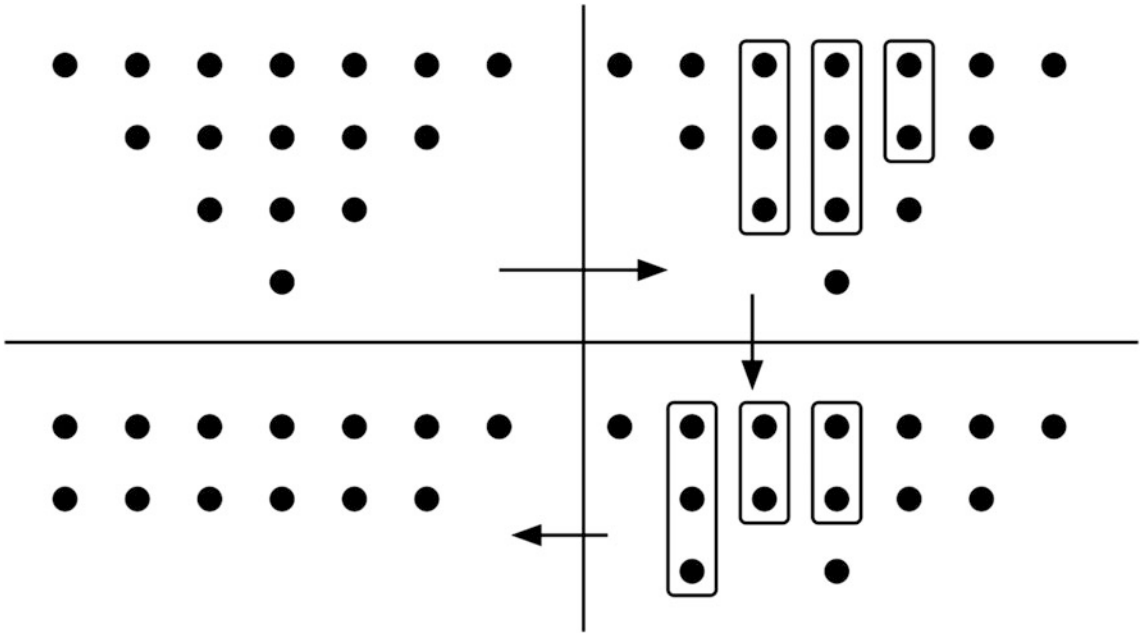

Fig. 11.27 Wallace tree reduction for 4 × 4 multiplier. In the top-right, HAs and FAs are used on positions 2, 3, and 4. On the bottom right, position 3 is only covered by an HA according to rule 4

Position 4 is combined using an HA because only two bits are available. Position 5 is combined using an FA because only an FA would allow it to end up with 2 bits.

The final result is shown in the lower left corner. There are two 6-bit words and none of the positions have three bits. Thus, the algorithm stops and the two words are fed into a fast adder (see Sect. 11.7) to produce the final product. Other than the final adder, the Wallace tree uses 3 FAs and 3 HAs. It also passes through two steps before reaching the final addition step.

Figures 11.28, 11.29 and 11.30 show the (simplified) Wallace trees for 5, 6, and 7-bit multipliers. The number of steps remains constant for 5 and 6-bits at 3. However, it increases to 4 steps for 7-bits. The number of steps is a critical factor in determining the delay of the Wallace multiplier. It is particularly important in determining how the delay scales for larger word length. The Wallace tree delay can be estimated as

$$t_{\text{mult}} = \text{steps} * t_{\text{sFA}} + t_{\text{adder}}$$

where tadder is the delay of the final adder. Since the tadder final delay can be optimized using fast adders, the critical factor in determining delay is the number of steps. The delay of Wallace tree multipliers is usually quoted as $O(\log N)$, and while this is true it is also a little misleading. Because 3:2 compressors are used, the number of stages is only of a logarithmic order and does not exactly follow a logarithmic function.

To find out how the delay behaves exactly, we can show that for any stage j, the number of rows (r) in the next stage $j + 1$ will be

$$r_{j+1} = 2 * \text{floor}\left(\frac{r_j}{3}\right) + r_j \mod 3$$

The number 3 appears in the equation owing to the 3:2 compressive nature of FAs. Starting at r_0, the initial number of rows, we can then predict the depth of the trees in all subsequent stages. The Wallace tree ends when the depth finally reaches 2, allowing the results to be fed to the final adder.

For example for the 4-bit adder in Fig. 11.27, $r_0 = 4$. We can calculate $r_1 = 3$ which can be confirmed from the figure. r_2 can be calculated and shown to be 2. Since we reached a depth of 2 in only two stages, then the Wallace tree is only two stages deep.

The same can be repeated for 6-bit with $r_0 = 6$, $r_1 = 4$, $r_2 = 3$, and $r_3 = 2$. This confirms there are only three stages for a 6-bit multiplier. For 7-bits $r_0 = 7$, $r_1 = 5$, $r_2 = 4$, $r_3 = 3$, and $r_4 = 2$ confirming the number of stages is 4. The number of stages is 4 for widths from 7 to 9. It is 5 for widths from 10 to 13, and 6 from 14 to 19. Thus, the number of stages does not grow exactly with the second logarithm of N, but it still grows in a sublinear and logarithmic order.

Finding a closed form for delay is difficult, but we can confirm by the observation that the extent of word lengths for which the number of stages remains constant increases

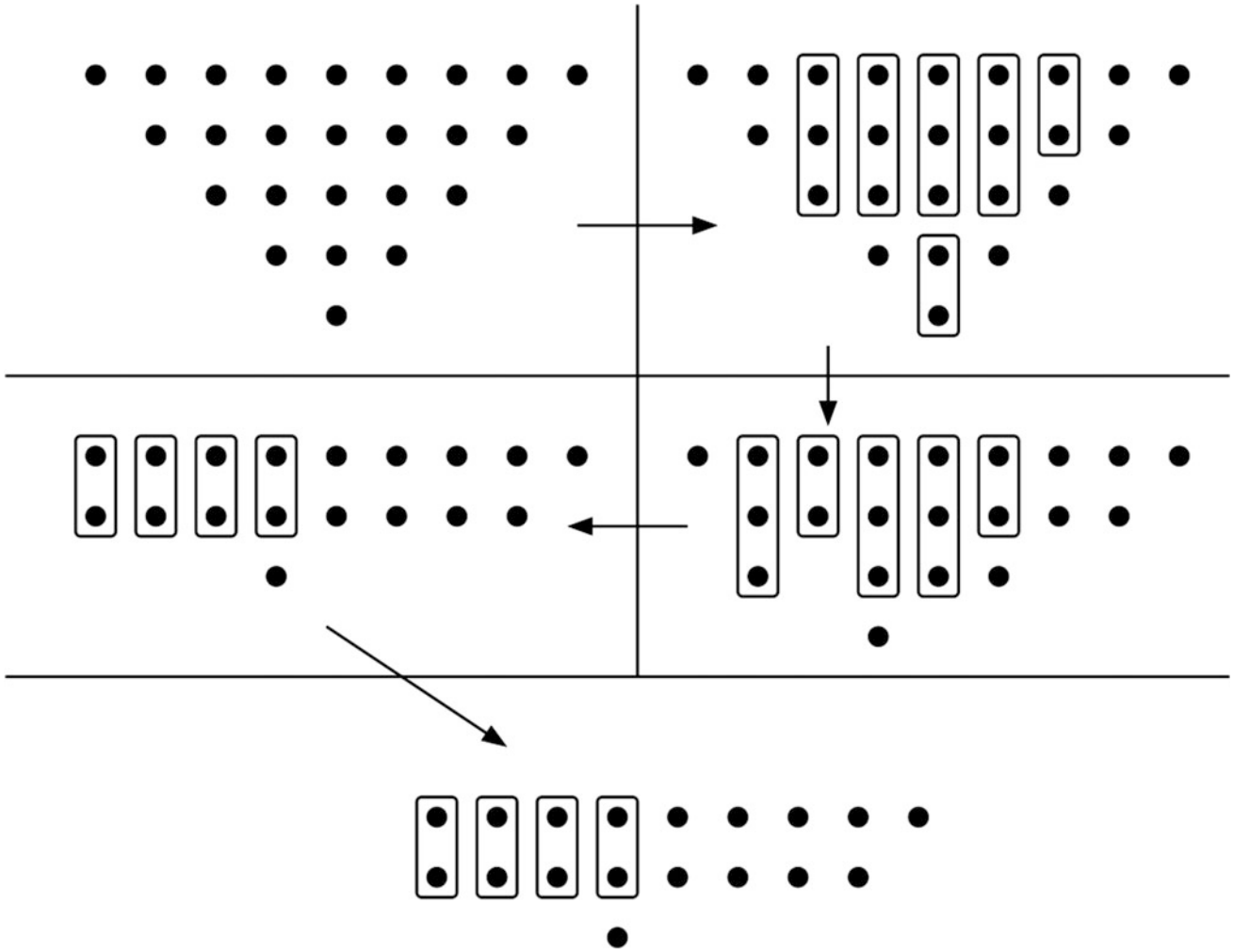

Fig. 11.28 5 bit Wallace tree

the larger the multiplier. Thus, the performance of the Wallace tree multiplier is significantly better than a straightforward array multiplier, especially for long word lengths. Surprisingly, Wallace trees are not larger than parallel multipliers. In fact, with judicious design choices, these multipliers can be smaller.

DADDA multipliers are very similar to Wallace tree multipliers. In fact, they follow pretty much the same algorithm. The main difference is that while the Wallace tree multiplier aims for maximum coverage in any single step, DADDA multipliers may under cover certain steps in order to reduce the total number of adders used. To do this while preserving or improving speed, DADDA multipliers use a more complicated algorithm to decide coverage.

DADDA depends on a sequence derived from the equation $dk + 1 = \text{floor}(1.5 * dk)$ and $d_1 = 2$. The sequence is thus: 2, floor(2 * 1.5), floor(floor(2 * 1.5) * 1.5), etc. Specifically the sequence is 2, 3, 4, 6, 9, 13, 19, 28, etc.

For an $N \times M$ multiplier, we choose the d which is less than the smaller of N and M, thus for a 3×8 multiplier the chosen d would be 2 because $2 < \min(3, 8)$. For a 4×4 multiplier d would be 3. For an 8×8 multiplier d would be 6.

For every stage, the aim is to try to reduce each column to be at or below the chosen d by using FAs and HAs. This is done as follows starting from LSB position and moving leftwards

- If the position has bits less than d, move on
- If the position has bits equal to $d + 1$, use an HA to compress two bits, pushing one result to the next position. The current position would be reduced to d, allowing us to move to the next bit position
- If the position has any higher number of bits, use an FA to compress, pushing bits and repeating again from the first step. In other words, this step would keep us looping on the same position until the height in the position reduces to d.

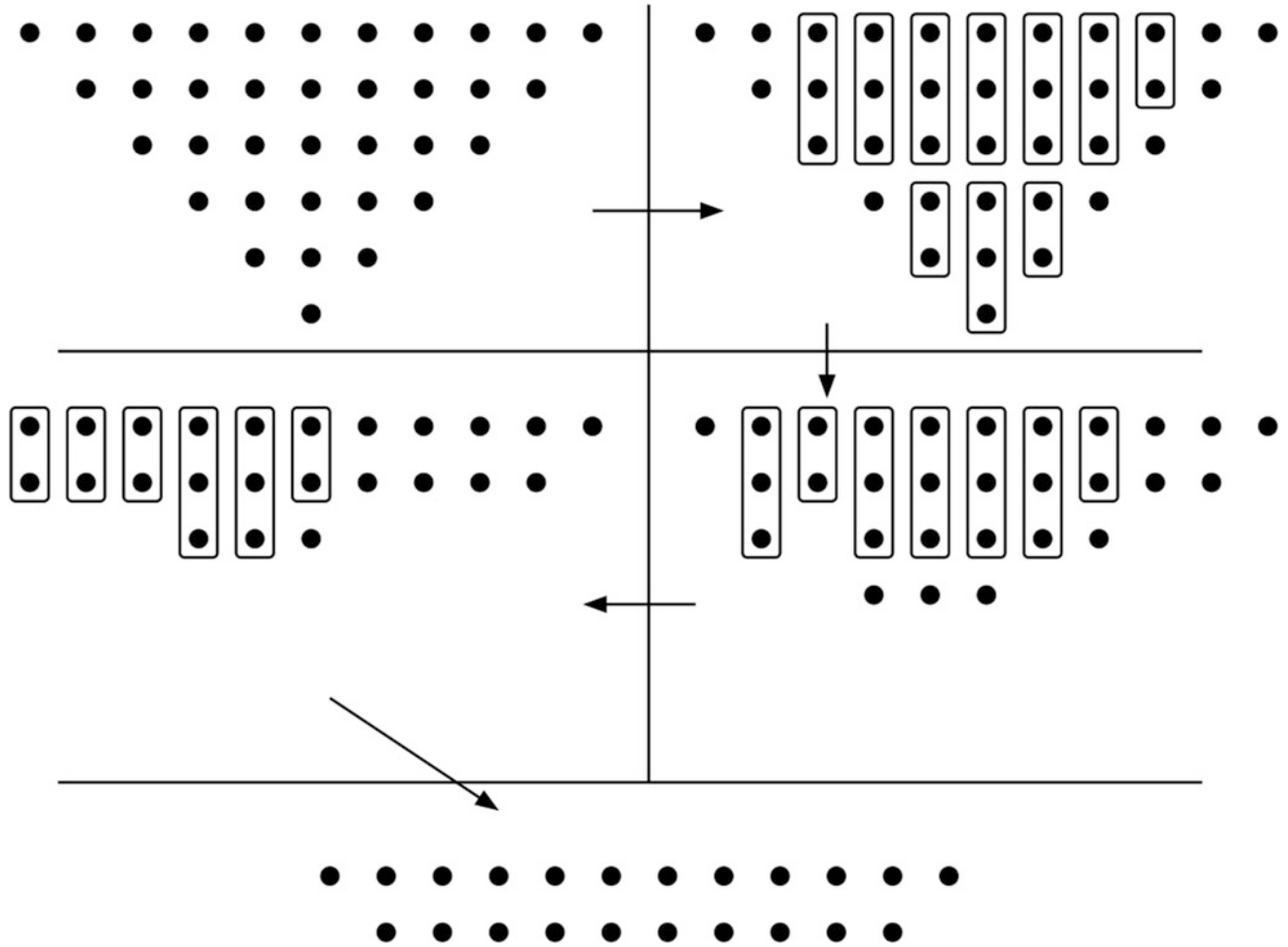

Fig. 11.29 6-bit Wallace tree

Once we move to the next stage, we use the next lower d and repeat again as above. This is repeated until there are only two bits left in all positions where the result is fed to a fast $N + M$ bit adder. We can already observe that this approach is very similar to Wallace trees, except it is more systematic and regular.

Figure 11.31 shows an 8×8 DADDA multiplier. In the first stage d is 6. Positions 0 through 5 have depths equal to or less than d and thus are left alone. Position 6 has 7-bits, thus only a half adder is used. In position 7 there are nine bits. Although there are only 8 drawn bits, there is one additional bit that came as a carry-in from position 6. Thus an FA is used reducing position 7 to 7 bits. An HA is needed to reduce position 7 to 6 bits, allowing us to move on to position 8. This is a major difference between Wallace trees and DADDA. When considering a position in DADDA, the number of bits in the position include all carries that come from the previous position in the same step.

Position 8 now has 9-bits, 7 drawn and two as carry-ins from the FA and HA in position 7. Thus, position 8 also needs an FA and HA to reduce it to 6 bits. Position 9 has 8-bits, 6 drawn and two carry-ins from position 8. It can be reduced to 6-bits using only a FA. Although position 10 has a carry-in coming from position 9, it still has only 6-bits, thus it receives no compressors.

In the second step, the next lower d is used. d is thus equal to 4. This allows us to move through positions 0 to 3 leaving them untouched. Position 4 has 5 bits, thus a single HA can reduce it to 4-bits. Position 5 has 6 bits, 5 drawn and 1 carry-in from position 4. This can be compressed to 4-bits using an FA and HA. Moving through the remaining positions, we reach the result shown in Fig. 11.31. Two more steps are needed before only two rows are left and $d = 2$.

DADDA multipliers can reduce the number of FAs and HAs used in reduction, but they result in denser final adders compared to Wallace tree multipliers. The number of steps needed can be deduced easily from the d sequence by counting the number of d's above 2 that need to be visited. For example for a 16-bit multiplier d is chosen to be 13 which would yield 6 steps.

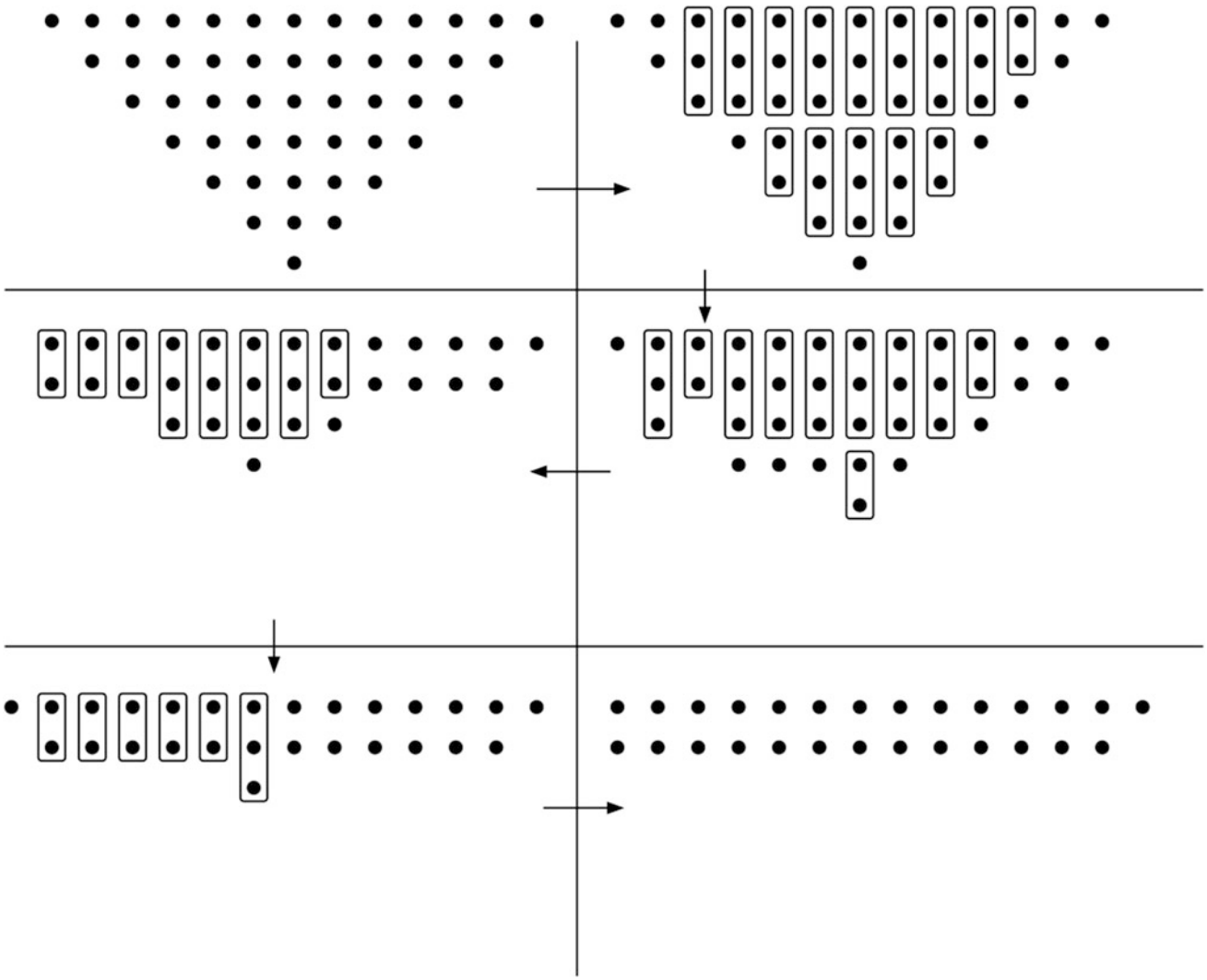

Fig. 11.30 7-bit Wallace tree

11.11 Booth Multiplication

1. Recall long strings of zeros help minimize constant multipliers
2. Understand how Booth recoding utilizes long strings of 1's
3. Use Booth recoding techniques to recode words
4. Prove the validity of Booth recoding
5. Use radix-4 Booth recoding to reduce complexity
6. Understand why radix-4 Booth multipliers never multiply by 3.

The Booth algorithm is a very useful way to visualize multiplication. Although the algorithm may initially seem more useful for software implementation, it can be used to implement very efficient hardware multipliers. Wallace tree and DADDA multipliers are often combined with Booth encoding to yield some of the best performing multipliers available.

There are two aspects to the Booth multiplication which are often confused with each other

- The first aspect is Booth encoding, or Booth recoding, which is a very useful way to re-state multipliers (the operands) in order to reduce the complexity of the multiplication operation
- Higher radix units, which can yield multipliers with fewer, albeit more complex building blocks.

The Booth algorithm is based on observations rooted in the constant multiplier. As discussed in Sect. 11.8, constant multipliers are simpler than general multipliers because some summands are null.

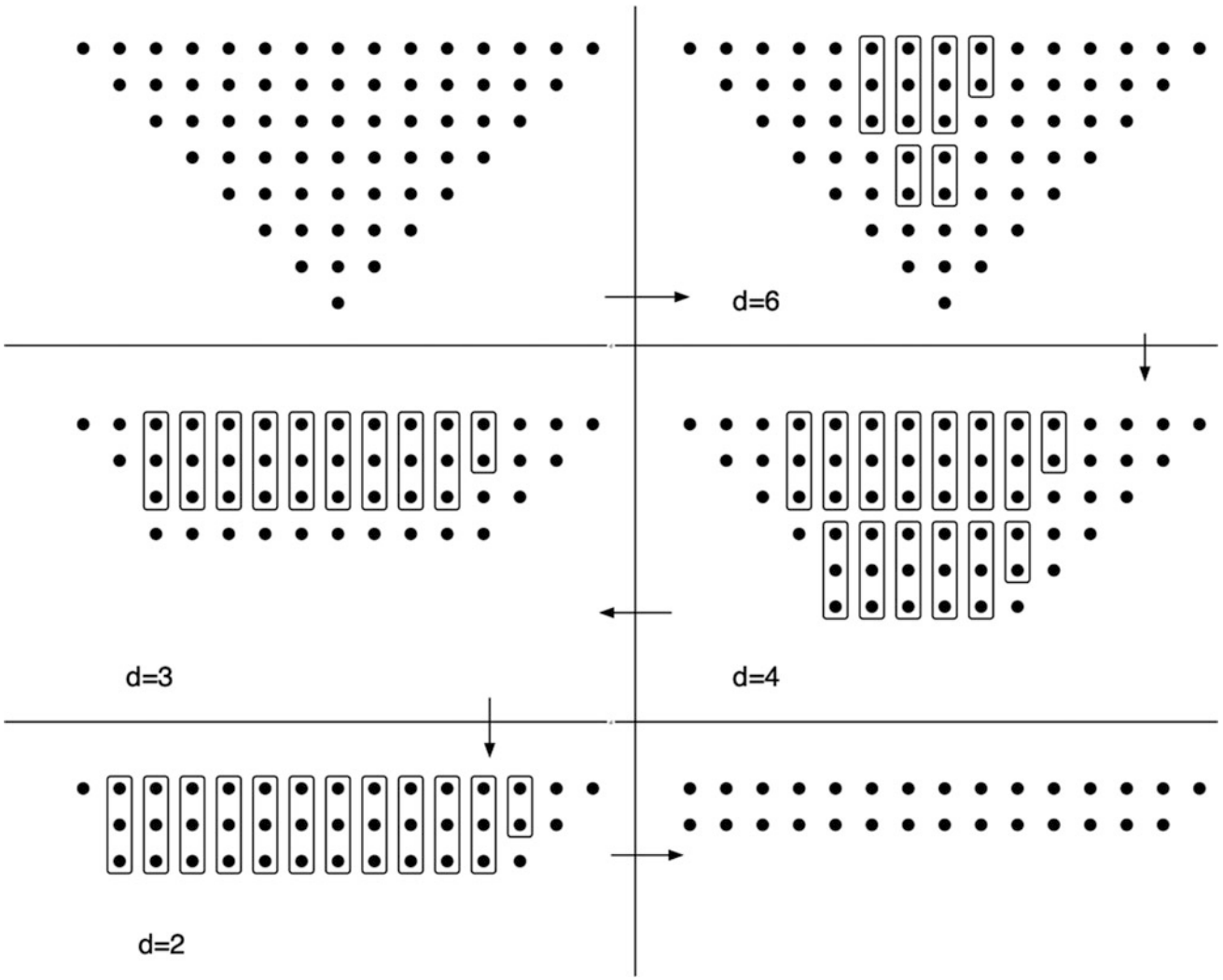

Fig. 11.31 8 × 8 DADDA multiplier

If we are multiplying by a constant coefficient, which of the following coefficients would yield the simplest constant multiplier: 4, 5, 16, 18, 19, 32, 33, 128, 256, 312, 512? The answer is of course 4, 16, 32, 128, 256, and 512. The reason is that these multipliers are powers of 2. Multiplication by these multipliers is thus only a single shift.

In other words all but one of the summands of the multiplication are null. This can be seen by writing these coefficients in binary form: 4 = "100", 16 = "10000", 32 = "100000", 128 = "10000000", 256 = "100000000" and 512 = "1000000000".

All the above multipliers share one aspect. They all have only one non-trivial bit, with the rest of the bits being null. This can be used to simplify multiplication even if the multiplier is not a power of 2. For example, the multiplier "100001000" is easy to implement as a constant multiplier since it has long sequences of zeros, yielding only few and isolated summands.

The main contribution of the Booth algorithm is that it also allows long strings of 1's to simplify multiplication. We might initially be tempted to think that multiplying by "1111" is the most complicated scenario in a 4 × 4 multiplier. After all, none of the summands are trivial. However, if we consider that this number is in two's complement, then we notice that "1111" is simply −1.

Thus multiplying by "1111" can be performed in one of two ways:

- Multiply the multiplicand by '1' four times and add all four summands
- Or just calculate the two's complement of the multiplicand.

Obviously, the second option is much simpler. This observation helps only when a number in two's complement format is all 1's. Booth's contribution is to extend this to cover long strings of 1's in the middle of a word

Consider for example the multiplier "1001110". There is an isolated string of three 1's within the multiplier. This string when isolated has the value ("1110") = 14. The string of three 1's has a value of 7, but because it is shifted once to the left its value is "1110" = 7 * 2 = 14.

The critical observation here is that 7 = 8 − 1. This allows us to reduce the long string of 1's into a single 1 and a single −1. Assuming that we use a modified binary notation where a position can have either of three values: 0, 1 and −1, the word "10001110" can be re-expressed as

$$100100 - 10$$

To confirm that this still represents the same number, notice that the three most significant bits are unchanged. The five least significant bits originally represented the value 14 = 01110. The modified five-bits can be calculated as

$$\begin{aligned} 100 - 10 &= 0 * 2^0 - 1 * 2^1 + 0 * 2^2 + 0 * 2^3 + 1 * 2^4 \\ &= 16 - 2 = 14 \end{aligned}$$

We can show that the same result applies for any long sequence of 1's isolated anywhere in a word. For example, in "10111110" the seven LSB's have the value "0111110" = 62. Recoding this by inserting a −1 in place of the LSB 1 in the string, and a 1 in the bit after the MSB 1: 10000 − 10. Now confirming that this still gives the same result

$$\begin{aligned} 1000 - 10 &= 0 * 2^0 - 1 * 2^1 + 0 * 2^2 + 0 * 2^3 + 0 * 2^4 + + 0 * 2^5 + + 1 * 2^6 \\ &= 64 - 2 = 62 \end{aligned}$$

The following questions are still unanswered

1. How can this recoding be done systematically.
2. How is multiplication performed after the recoding of the multiplier (operand).
3. Why is this useful.
4. How can we prove that this works for any long string of 1's.

Recoding is done by dividing any multiplier into sequences of overlapping pairs of bits. Each pair is then encoded into either 0, 1, or −1. Because we are covering overlapping pairs, we end up with the same number of bits even if we are replacing pairs of bits by single bits.

The rules used for encoding are shown in Table 11.6. The method used for recoding is best illustrated with an example. Consider for example the string "1000111100". As shown in Fig. 11.32, the recoding process begins by adding a zero below the LSB. This zero exists below the binary point and should be included in all cases. We need to start coverage at this bit for reasons that will become clear soon.

Starting at the additional zero in the LSB and moving from right to left, overlapping pairs of bits are considered. Each pair is translated according to Table 11.6. This will result in a word of the same length which represents the same number. For example, the first pair of bits is "00" which is translated into 0. The second pair is also "00", translated into 0. The third pair is "10" translated into −1. The following pair is "11", translated into 0, and so on.

It is useful to consider the reasoning behind this recoding. Taking overlapping pairs allows us to observe bit transitions. Two zeroes in sequence mean there is no bit transition happening from the lower bit to the higher bit. Thus, the multiplicand need not be multiplied. When we observe the sequence "10", then this could be the start of a sequence of 1's. Thus, this gets translated into a −1 in the lower position bit. This can be confirmed from the numerical examples discussed earlier.

A sequence "11" indicates we are already in the middle of a string of 1's. Since we already covered the transition 10, then we can safely assume the "11" pair will result in a 0 and will have no effect on the multiplicand. *A* sequence "01" indicates a sequence of 1's is ending, this necessitates that a 1 is included in place of the 0, as was illustrated in the earlier numerical examples.

Figure 11.33 shows the recoding of a longer and more complicated word. The main difference here is that there are two long strings of 1's instead of just 1. The method and its application, however, remain unchanged.

Figure 11.34 shows the recoding of "11110". This example illustrates that even if the string of 1's extends to the end of the word, the recoding approach still works. The reason is that if the string of 1's does not end, this means the word is sign-extended and represents a negative number. Thus, the −1 inserted when the string of 1's starts is sufficient without a corresponding +1 inserted at a higher position.

Figure 11.35 shows an example that illustrates the need to include the "0" beyond the binary point. If we attempt to recode this word without inserting the additional "0", we would observe a sequence of "11" immediately and would recode as follows: "00111" = "01000", which gives the wrong result. If we also consider the additional "0" the recoding becomes: "0111" = 0100 − 1 = 7, which is the correct result. Thus, the zero before the binary point is necessary to recognize a one string that start at the beginning of the word.

Figure 11.36 illustrates one final example of recoding. In this case, each 1 indicates the start of a sequence. However, because it is an isolated 1, the very next pair of bits indicate an end of a sequence. Thus, the recoding ends up as shown in the figure.

Table 11.6 Booth encoding scheme

Bit pair	Encoding
"00"	0
"01"	1
"10"	−1
"11"	0

Fig. 11.32 Overlapping pair coverage of the word "1000111100"

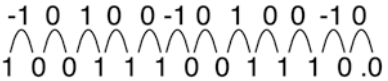

Fig. 11.33 Recoding of "100111001110"

Multiplication is performed by first recoding the multiplier. After this is done, the multiplicand for each bit of the multiplier is either shifted and added, shifted, and subtracted, or does nothing. This is best illustrated with an example.

Consider the multiplication "01111" * "01110". In decimal, this multiplication is 15 * 14 = 210. If we consider "01111" to be the multiplicand and "01110" to be the multiplier, then the first step is to recode "01110". Using the rules in Table 11.6, "01110" is recoded as 100 − 10. Now the multiplication is performed either by long multiplication or successive multiplication. Using long multiplication

0	1	1	1	1					
1	0	0	–1	0					
				0	0	0	0	0	
			– 0	1	1	1	1		
		0	0	0	0	0			
	0	0	0	0	0				
0	1	1	1	1					

It is important to notice that in the recoded form, the 1 in the MSB does not represent a sign bit. It represents a large positive magnitude.

Booth recoding is extremely useful because it increases the number of nulls in a multiplier. This reduces the number of summands, thus making the calculation of the product easier. Booth recoding does this by reducing long strings of 1's into a single +1 and a single −1. The +1 represents an addition, the −1 represents a subtraction. Subtraction can be performed using the same hardware as addition. For example, adding inverters for one of the operands and feeding C_{in} a "1" in a ripple carry adder calculates the two's complement for that operand. Thus, transforming the N-bit adder into a subtractor. In other words, subtractors have the exact same hardware cost as adders.

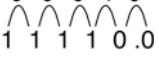

Fig. 11.34 Recoding of "11110"

Fig. 11.35 Recoding of "00111"

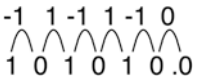

Fig. 11.36 Recoding of "101010"

In the example illustrated above multiplying by "01110" would have generated three summands. In the Booth recoded form, the multiplier operand 100 − 10 generated only two summands.

We can already conclude that Booth recoding is most useful when there are long strings of 1's. For example "01111110" would generate 6 summands compared to only two in its recoded form. The longer the sequence of 1's, the more effective the recoded form.

In a straightforward multiplication, the worst case is when the multiplier operand is all 1's. In Booth recoded form an all 1's multiplier operand would produce a very efficient result. So what is the worst case for Booth recoding? Evidently, the worst case is when there is an interleaving of 1's and 0's as in Fig. 11.36. This kind of sequence produces as many summands as there are bits in the multiplier. This is worse than straightforward multiplication where about half of the summands are active.

So does this indicate that Booth recoding is counter-productive? No, because, especially for larger multipliers, the probability of a perfectly interleaved multiplier operand is much smaller than the probability of a multiplier where long sequences of 1s and 0s exist. Thus, on average Booth recoding leads to a net gain by reducing the average number of summands.

The validity of Booth recoding for an arbitrarily long sequence of 1's in an arbitrary position can be proved as follows. Assume we have a string of 1's within a word

starting at bit position m and ending at bit position k. Note that this proof does not necessitate that the sequence of 1's is unique within the word. The LSB of the 1 string is at position m and the MSB is at position k. The number of 1's in the string is thus $n = k - m + 1$.

Calculating the value of the 1's string:

$$\text{value} = 2^k + 2^{k-1} + 2^{k-2} + \cdots + 2^{m+1} + 2^m$$

This is a geometric progression whose base is 2^m and the common ratio is 2. Calculating the sum of the progression, which is the value of the string

$$\text{value} = \frac{2^m(1-2^n)}{1-2} = 2^{n+m} - 2^m$$

But recalling $n = K - m + 1$

$$\text{value} = 2^{n+m} - 2^m = 2^{k-m+1+m} - 2^m = 2^{k+1} - 2^m$$

Thus, the value of the long string of 1's can be replaced with a +1 at the position $k + 1$ and a −1 at the position m. This is the rule used in Table 11.6.

The modified Booth algorithm is a subset of a much larger domain of arithmetic where the radix of operations is higher than 2. In the modified Booth algorithm we assume we have a building block that can handle more complicated operations than the FA. This helps us significantly reduce the number of operations required to reduce the summands. On the other hand, we have to be aware of the complexity of each of these operations.

The modified Booth algorithm assumes that Booth recoding has already been performed. It also assumes that the result of the Booth recoding has an even number of bits. If the original word has an odd number of bits, then it should be sign-extended to make it even.

In the recoded word, each pair of bits (non-overlapping) in the multiplier operand is compressed into a single operation. Because the operation includes two bits, it runs the possibility of being more than just addition or subtraction. However, the number of operations is reduced by half. This is best illustrated through examples.

Consider the example where the multiplier operand is "0111100". Here the number of bits is not even, so we need to sign extend the word by 1 bit into "00111100". When Booth recoded this becomes 01000 − 100. Note the insignificant additional "0" in the MSB, this is necessary to maintain an even number of bits. This recoded word is then divided into pairs of bits: (01)(00)(0 − 1)(00). Each pair of bits is compressed into one operation. For example 00 means that nothing happens to the multiplicand. (0 − 1) means the multiplicand is multiplied by −1. (01) is multiplied by +1. Thus the multiplier operand is compressed into 1 0 − 1 0.

Note that the multiplier operand is now half its original size, this means there will be half as many summands. However, because the radix of the multiplier is now increased to radix-4, then shifting needs to be increased. In other words, in radix-2 multiplication each successive bit in the multiplier causes the summand to be shifted by 1 bit to the left. In this case, however, the shift is by two bits for each entry in the multiplier. In other words, if M is the multiplicand and P is the product then:

$$\begin{aligned} P = M * (10 - 10) &= 1 * M * 4^3 + 0 * M * 4^2 - 1 * M * 4^1 \\ &\quad + 0 * M * 4^0 \\ &= 1 * M * 2^6 + 0 * M * 2^4 - 1 * M * 2^2 + 0 * M * 2^0 \end{aligned}$$

This is not a very radical change. All we are doing is just shifting each summand by two bits relative to the previous summand instead of a single bit. However, the price for this higher radix approach lies elsewhere.

Consider an example where the multiplier is "011110". The number of bits is even, so no sign extension is necessary. When Booth recoded, the word becomes 1000 − 10. Dividing this into pairs, we obtain (10)(00)(−10). What is 10? It can be translated as multiplication by 2. Similarly, −10 is multiplication by −2. Thus the multiplier operator is translated into (2)(0)(−2). The building block that replaces the FA needs to be able to either add or subtract the multiplicand or to multiply it by 2 then add or subtract it. Since multiplication by 2 is easy, this price is normally worth paying in order to reduce the number of summands.

To summarize the modified Booth algorithm works as follows

1. Sign extend the multiplier to have an even number of bits.
2. Booth recode the multiplier.
3. Divide the multiplier into pairs of bits.
4. Each pair of bits is translated according to Table 11.7.
5. The number of operations required to calculate the product is now reduced by half.
6. However each operation in 5 above is now more complex. Instead of simple addition, we need to be able to add subtract and shift by 2 or any combination thereof.

Each Booth recoded bit has three possibilities: 0, 1, and −1. Thus, any sequence of two bits *should* have 9 possible combinations. However, Table 11.7 has only seven entries, it seems to be missing two entries, specifically: "11" and "−1 −1". These two possibilities would be translated into 11 = 2 + 1 = 3 and −1−1 = −2−1 = −3. These two possibilities require the building block to multiply by 3, which is significantly more complicated than multiplying by 2. In fact, it would push us back into a normal radix-2 multiplier. This suggests the modified Booth algorithm is futile.

Fortunately the two possibilities "11" and "−1−1" cannot occur, which is why they are not shown in Table 11.7.

Table 11.7 Modified Booth encoding scheme

Booth recoded pair	Modified compressed multiplier
"00"	0
"01"	1
"0–1"	−1
"10"	2
"−10"	−2
"1–1"	2 − 1 = 1
"−11"	−2 + 1 = 1

Notice that a −1 in Booth recoding occurs when entering a sequence of 1's, a +1 occurs when exiting a sequence of 1's. Two −1's in sequence means that we enter two sequences of 1's in series. This is impossible to happen without an intermediate +1. We cannot enter a new sequence of 1's without first exiting the first sequence of 1's, thus two −1's in sequence cannot occur.

Similarly, two 1's in sequence means we are exiting two sequences of 1's in series. This cannot happen because we cannot exit a second sequence of 1's without first entering another sequence of 1's. Thus any two +1's must be separated by at least an intervening −1 and vice versa.